

Personal Media Catalog - Project Instructions

Overview

Due Date: [January 10, 2026]

Weight: 20% of Final Grade

Team Size: 2-3 students recommended

You will build a **Python console application** to manage a personal media catalog (for books, movies, video games, etc.). The application will use **XML for data storage** and must be developed collaboratively using **Git and GitHub**. This project will test your skills in XML technologies, Python programming, software design, and version control.

Core Requirements

Your application must fulfill the following technical requirements:

1. Data Persistence with XML:

- All media data must be stored in an XML file (`catalog.xml`).
- The structure of this XML file must be defined and validated by an **XML Schema** (`media_catalog.xsd`).

2. Python Application with XML Parsing:

- The main application must be written in Python.
- You must use **Python's built-in `xml.etree.ElementTree` module** to read the initial catalog from `catalog.xml` when the program starts and write any changes back to the file when the program exits or when a "save" command is executed.

3. Mandatory Design Patterns:

- **Factory Pattern:** For creating different types of media objects (e.g., `Book`, `Movie`, `VideoGame`).
- **Singleton Pattern:** For the main catalog manager class to ensure only one instance exists.
- **Repository Pattern:** (Python equivalent of DAO) To separate the business logic from the XML persistence logic.

4. XSLT Reporting:

- Implement a feature that allows the user to generate an HTML report from the `catalog.xml` file.
 - You must create at least **two different XSLT stylesheets** (e.g., `full_catalog.xsl`, `movies_only.xsl`) that produce different HTML views of the data.
 - Use Python's `lxml` library or other XSLT processors for the transformation.
5. **Version Control with Git & GitHub:**
- Your team must use a **GitHub repository** for collaboration.
 - Your commit history must show a steady, logical progression of work from all team members.
-

Project Deliverables & Timeline

Phase 1: Project Setup & Data Modeling (Due: December 10, 2025)

1. **Form Your Team** and decide on initial roles.
2. **Create a GitHub Repository:**
 - One team member creates a new **GitHub Organization**, then creates the project repository within it.
 - Invite all other team members and your instructor as collaborators.
3. **Set up your `README.md`** with the project title and description.
4. **Create an XML Schema (`media_catalog.xsd`):**
 - Design your schema to be flexible. It should define a `<catalog>` that contains various `<media>` items.
 - Use XML Schema complex types and inheritance. For example, create a base `MediaType` and then extend it with `BookType`, `MovieType`, etc., each with their own specific elements.
 - Commit and push your XSD file to the `main` branch.

Phase 2: Core Application Development (Due: December 20, 2025)

1. **Set up your Python project** with a proper structure:

```
media_catalog/
├── src/
│   ├── __init__.py
│   ├── main.py
│   ├── models/
│   ├── services/
│   └── repositories/
├── data/
│   ├── catalog.xml
│   └── media_catalog.xsd
├── requirements.txt
└── README.md
```

2. **Implement the Design Patterns:**

- **Repository Pattern:** Create a `CatalogRepository` class that handles all `load_from_xml()` and `save_to_xml()` operations using `xml.etree.ElementTree`.
- **Singleton Pattern:** Create a `CatalogManager` class that uses the Singleton pattern. This class will hold the in-memory list of media items and use the `CatalogRepository`.
- **Factory Pattern:** Create a `MediaFactory` class with a method like `create_media(media_type, **kwargs)` that returns a `Book`, `Movie`, etc., object.

3. **Implement the Console UI Menu** with options like:

```
def show_menu():
    print("\n=== Personal Media Catalog ===")
    print("1. Add Media")
    print("2. View All Media")
    print("3. Search Media")
    print("4. Delete Media")
    print("5. Generate HTML Report")
    print("6. Save and Exit")
```

4. **Work in Feature Branches!** Create branches like `feature/repository-implementation` or `feature/factory-pattern`.

Merge them via Pull Requests.

Phase 3: Advanced Features & Reporting (Due: December 29, 2025)

1. Develop your XSLT Stylesheets:

- Create at least two `.xsl` files (e.g., `catalog_to_html.xsl`, `wishlist.xsl`).
- They should transform your `catalog.xml` into different HTML formats.

2. Integrate XSLT into your Python App:

- Install the `lxml` library (add to `requirements.txt`).
- Add a menu option for generating reports.
- When selected, use `lxml` to transform the `catalog.xml` using one of your stylesheets and save the output as an `.html` file.
- The user should be able to open this file in a web browser.

Phase 4: Final Submission (Due: January 10, 2026)

1. **Final Code Push:** Ensure all your final code is in the `main` branch of your GitHub repository.
 2. **Complete the Documentation:**
 - Update your `README.md` with clear instructions on how to install dependencies and run your project.
 - In a `docs/` folder, include a `FINAL_REPORT.md` that briefly explains:
 - Team member contributions.
 - How and why you implemented each design pattern.
 - Challenges you faced and how you overcame them.
 3. **Prepare for Demo:** Be ready to give a 5-minute live demonstration of your working application.
-



Getting Started: A Step-by-Step Guide

1. **Kick-off Meeting:** Discuss what media types you will support (e.g., Book, Movie, Game). Sketch the data attributes for each on paper.
2. **Schema First:** Write your `media_catalog.xsd` first. This defines your contract. Validate a sample `catalog.xml` against it using an online validator.
3. **Skeleton Code:** Create the basic Python class structure:

```
# models/media.py
from abc import ABC

class Media(ABC):
    def __init__(self, title, year):
        self.title = title
        self.year = year

class Book(Media):
    def __init__(self, title, year, author, pages):
        super().__init__(title, year)
        self.author = author
        self.pages = pages
```

4. **Tackle Persistence:** Start by having your `CatalogRepository.load_from_xml()` method parse the XML and print the contents to the console. Then work on `save_to_xml()`.
 5. **Build the UI Last:** Once your core logic (adding, deleting, saving, loading) works, build the console menu around it.
-

Python-Specific Technical Notes

Required Libraries:

- `xml.etree.ElementTree` (built-in) - for XML parsing
- `lxml` - for XSLT transformations and advanced XML handling
- Add these to your `requirements.txt`

Example Code Snippets:

Singleton Pattern in Python:

```
class CatalogManager:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Factory Pattern:

```
class MediaFactory:
    @staticmethod
    def create_media(media_type, **kwargs):
        if media_type == "book":
            return Book(**kwargs)
        elif media_type == "movie":
            return Movie(**kwargs)
        # ... other media types
```

XML Writing with ElementTree:

```
import xml.etree.ElementTree as ET

def save_to_xml(catalog, filename):
    root = ET.Element("catalog")
    for media in catalog:
        media_elem = ET.SubElement(root, "media")
        ET.SubElement(media_elem, "title").text = media.title
        # ... add other elements
    tree = ET.ElementTree(root)
    tree.write(filename)
```



Grading Rubric

Category	Weight	What We're Looking For
Functionality	30%	The app runs without crashes. All menu options work as expected. Data is persisted correctly between runs.
XML & Python Parsing	25%	A valid, well-designed XML Schema. Correct use of ElementTree for all read/write operations.
Software Design	20%	Correct implementation of Factory, Singleton, and Repository patterns. Clean, Pythonic, and modular code.
XSLT Reporting	10%	Two functional XSLT stylesheets that correctly transform the XML into distinct, viewable HTML reports.
Version Control	10%	Meaningful commit messages, use of feature branches and pull requests, and contributions from all team members.
Documentation & Demo	5%	A clear README, a thoughtful final report, and a smooth, professional demonstration.



Tips for Success

- **Communicate!** Use Discord, WhatsApp, or meet regularly.
- **Start Early.** The XML parsing and design patterns can be tricky.
- **Use Virtual Environments:** Create a virtual environment and track dependencies in `requirements.txt`.
- **Commit Often.** Small, frequent commits are better than one massive dump at the end. "Fixed a typo" is a bad message. "Repository: implemented load_from_xml method" is a good one.
- **Test Relentlessly.** After every new feature, test the entire workflow: add item -> save -> exit -> restart -> view item.

Good luck, and have fun building your catalog with Python!